

Дополнительные главы дискретной математики:

Алгоритмы и структуры данных

Лекция 4. Красно-чёрные деревья

Ключарёв Пётр Георгиевич

pgkl@yandex.ru

twitter.com/klyucharev

Кафедра ИУ8 «Информационная безопасность»
МГТУ им. Н.Э. Баумана

2013 г.

Красно-чёрное дерево

Красно-чёрное дерево

Красно-чёрное дерево (red-black tree) — это двоичное дерево поиска, вершины которого разделены на красные (red) и чёрные (black). Таким образом, каждая вершина хранит один дополнительный бит — её цвет.

Поля

Каждая вершина красно-чёрного дерева имеет поля *color* (цвет), *key* (ключ), *left* (левый ребёнок), *right* (правый ребёнок) и *p* (родитель).

Если у вершины отсутствует ребёнок или родитель, соответствующее поле содержит *nil*.

Будем считать, что значения *nil*, хранящиеся в полях *left* и *right*, являются ссылками на фиктивные листья дерева. Каждая содержащая ключ вершина имеет двух детей.

Красно-чёрное дерево

Свойства

Потребуем, чтобы красно-чёрное дерево удовлетворяло следующим свойствам:

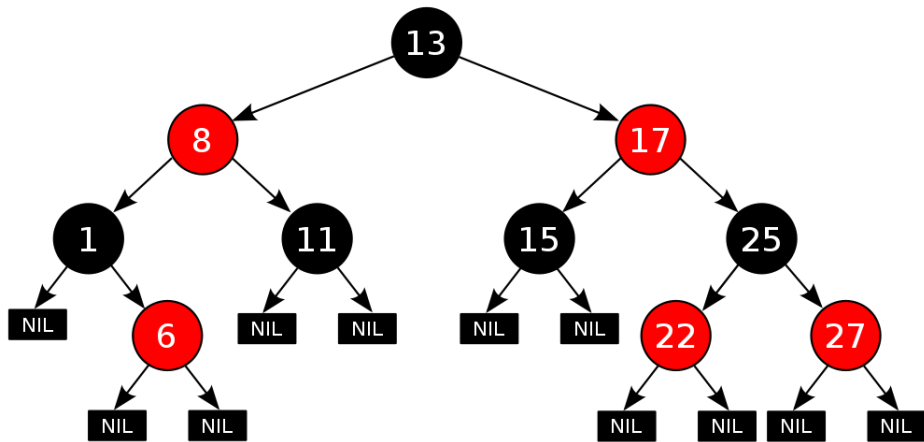
- 1 Каждая вершина — либо красная, либо чёрная.
- 2 Каждый лист — чёрный.
- 3 Оба ребенка красной вершины — чёрные.
- 4 Все пути, идущие вниз от корня к листьям, содержат одинаковое количество чёрных вершин.
- 5 Корень дерева всегда чёрный

Чёрная высота

Число чёрных вершин в любом поддереве вершины x называется чёрной высотой этой вершины $bh(x)$.

Чёрной высотой дерева называется чёрная высота его корня.

Пример красно-чёрного дерева



Высота

Теорема

Поддерево с корнем в x содержит по меньшей мере $2^{bh(x)} - 1$ внутренних вершин.

Доказательство.

Для листьев чёрная высота равна 0 и утверждение выполняется. Пусть теперь вершина x не является листом и имеет чёрную высоту k . Тогда оба её ребёнка имеют чёрную высоту не меньше $k - 1$ (красный ребёнок будет иметь высоту k , чёрный — $k - 1$). По предположению индукции левое и правое поддерева вершины x содержат не менее $2^{k-1} - 1$ вершин, и потому поддерево с корнем в x содержит по меньшей мере $2^{k-1} - 1 + 2^{k-1} - 1 + 1 = 2^k - 1$ внутренних вершин. □

Высота

Теорема

Красно-чёрное дерево с n внутренними вершинами имеет высоту не больше $2 \log_2(n + 1)$.

Доказательство.

Обозначим высоту дерева через h . Согласно свойствам красно-чёрного дерева, не менее половины вершин на пути от корня к листу, не считая корень, составляют чёрные вершины. Поэтому, чёрная высота дерева не меньше $h/2$. Тогда

$$n \geq 2^{h/2} - 1.$$

Логарифмируя, получим $\log_2(n + 1) \geq h/2$, или $h \leq 2 \log_2(n + 1)$. $\square$

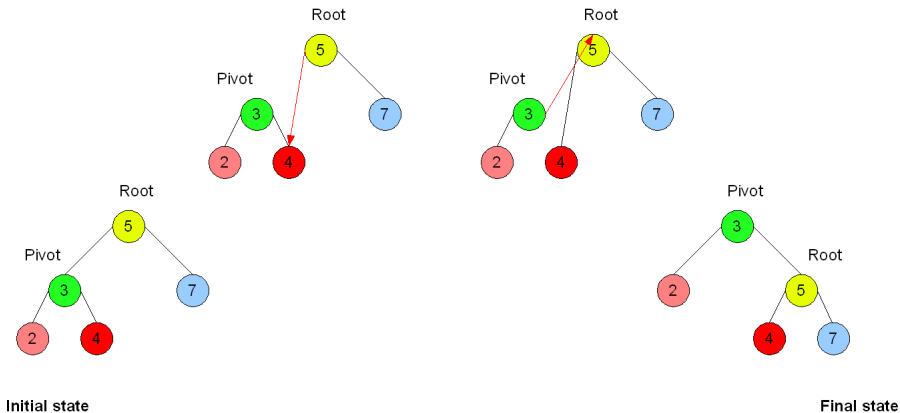
Поиск и другие операции доступа к данным

Операции:

- Search,
- Minimum,
- Maximum,
- Successor,
- Predecessor

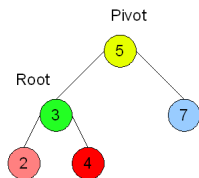
Ничем не отличаются от этих операций в обычном дереве поиска.
Сложность этих операций $O(h) = O(2 \log_2(n + 1)) = O(\log n)$

Правое вращение

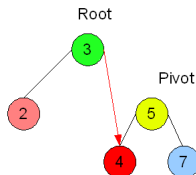
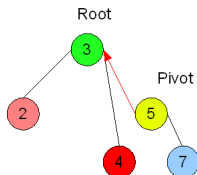
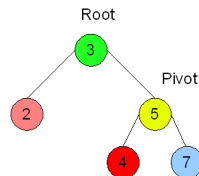


Левое вращение

Final state



Initial state



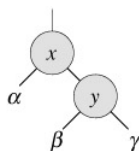
Левое вращение

Алгоритм 1 Левое вращение

```

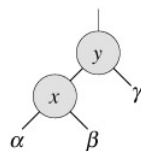
1: procedure
   LEFTROTATE( $T, x$ )
2:    $y := x.right$ 
3:    $x.right := y.left$ 
4:   if  $y.left \neq nil$  then
5:      $y.left.p := x$ 
6:    $y.p := x.p$ 
7:   if  $x.p = nil$  then
8:      $T.root := y$ 
9:   else if  $x = x.p.left$  then
10:     $x.p.left := y$ 
11:   else
12:     $x.p.right := y$ 
13:    $y.left := x$ 
14:    $x.p := y$ 

```



LEFT-ROTATE(T, x)

RIGHT-ROTATE(T, y)



Добавление вершины

Концепция

Добавление вершины осуществляется при помощи процедур работы с двоичным деревом поиска, вращений и перекрашиваний вершин.

Заметим

Заметим, что:

- Свойство 2 (Все листья черные) выполняется всегда.
- Свойство 3 (Оба ребенка красной вершины — черные) может нарушиться только при добавлении красной вершины, при перекрашивании черной вершины в красный цвет или при повороте.
- Свойство 4 (Все пути от любой вершины до листовых вершин содержат одинаковое число черных вершин) может нарушиться только при добавлении черной вершины, перекрашивании вершины, или при повороте.

Добавление вершины

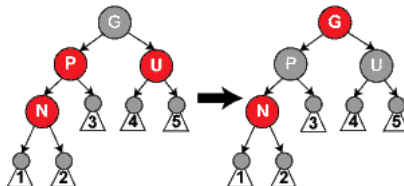
Обзор алгоритма

- 1 Вставляем вершину как обычно (Insert).
- 2 Делаем вставленную вершину красной.
- 3 Производим коррекцию для вставленной вершины.
- 4 Красим корень дерева в черный цвет.

Случай 1

Отец текущего узла черный, то есть Свойство 3 (Оба ребенка красной вершины — черные) не нарушается. Остальные свойства — не нарушаются. Дерево корректно.

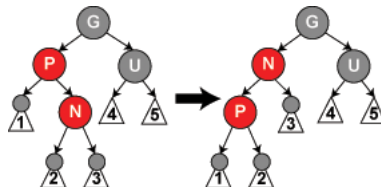
Добавление вершины



Случай 2

Если и отец и дядя — красные, то они оба могут быть перекрашены в черный, а дедушка — в красный (для сохранения свойства 4). Теперь у текущего красного узла черный родитель. Так как любой путь через родителя или дядю должен проходить через дедушку, число черных узлов в этих путях не изменится. Однако, дедушка теперь может нарушить свойство 3 (оба ребенка красной вершины — черные). Поэтому процедура коррекции должна выполняться для дедушки.

Добавление вершины

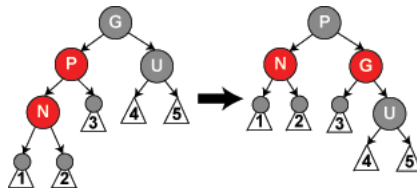


Случай 3

Отец — красный, а дядя — черный. Текущий узел — правый потомок отца, а отец — левый потомок деда.

Производится левое вращение. Свойство 3 (Оба ребенка красной вершины — черные) все еще нарушено. Изменяется положение двух красных узлов, поэтому Свойство 4 не нарушается.

Добавление вершины



Случай 4

Отец — красный, а дядя — черный, текущий узел — левый потомок отца, а отец — левый потомок деда. В этом случае выполняется правое вращение. В результате получается дерево, в котором бывший родитель теперь является родителем и текущего узла и бывшего дедушки.

Дед является черным, так как отец — красный. Поэтому цвета бывшего отца и бывшего деда надо поменять. Все свойства выполняются и дерево корректно.

Добавление вершины

Случаи 3' и 4'

Если отец — правый потомок деда, то в случаях 3 и 4 следует заменить право на лево, а лево на право.

Сложность

$O(\log_2 n)$

Удаление

Замечание

При удалении вершины из двоичного дерева поиска, мы всегда удаляем вершину с не более чем одним нелистовым ребенком.

Простой случай 1

Удаляемая вершина красная. Заменяем ее на нелистового ребенка.

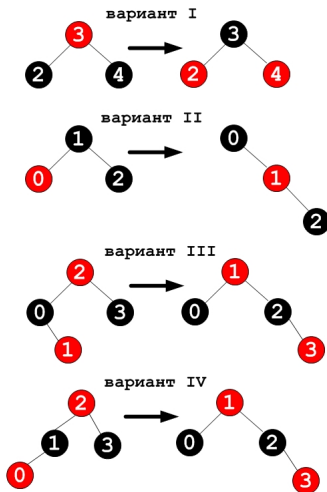
Простой случай 2

Удаляемая вершина черная, но ее нелистовой потомок — красный. Заменяем удаляемую вершину на ее нелистового ребенка и перекрашиваем его в черный цвет.

Остальные случаи

Сводим к одному из простых случаев путем замены цветов, когда это возможно и следующих преобразований.

Удаление вершины: сложные случаи



Упражнения

- 1 Покажите, что самый длинный путь вниз от вершины x к листу не более чем вдвое длиннее самого короткого такого пути.
- 2 Покажите, что вращения сохраняют свойство упорядоченности.
- 3 Покажите, что произвольное двоичное дерево поиска с n вершинами может быть преобразовано в любое другое дерево с тем же числом вершин (и теми же ключами) с помощью $O(n)$ вращений.
- 4 При добавлении вершины, мы красим новую вершину в красный цвет. Почему мы не можем ее покрасить в черный цвет?
- 5 Нарисуйте красно-чёрные деревья, которое получаются при последовательном добавлении к пустому дереву ключей 41, 38, 31, 12, 19, 8.

И ещё упражнения

- 1 Рассмотрим красно-чёрное дерево, полученное добавлением n вершин к пустому дереву. Убедитесь, что при $n > 1$ в дереве есть хотя бы одна красная вершина.
- 2 Напишите процедуру `RightRotate`.
- 3 Пусть a , b и c — произвольные вершины в поддеревьях α , β и γ на рисунке. Как изменится глубина a , b и c при выполнении левого вращения?

